

NyayaSathi × Sanhita

The Master Plan — Launch in 21 Days, Eval-Gated, Built to Last

A two-sided legal-aid platform for India:

free citizen-facing voice helpline that converts to verified-lawyer matches and a research workbench priced for solo lawyers, small firms, and enterprise compliance teams.

What this document is

The single source of truth for what we are building, why this design, what we deliberately choose *not* to build, and what each day of the next 21 days looks like. Every commit, eval result, and architectural decision should be traceable back to a section of this plan. Re-generate from `docs/generate_master_plan.py` any time the plan changes.

Status: **Sprint 0, Day 1.** Eval baseline being authored.

Table of Contents

0	Executive verdict in one page	3
1	Sanhita codebase audit — integrate, don't rebuild	4
2	The three-product reality	7
3	The four customer journeys	9
4	The architectural commitment — what AI does NOT do	11
5	Multi-modal data: storage and upload	13
6	The three stages of casework: Research → Analysis → Strategy	15
7	New surfaces: Predictive · Compliance · Contract Intelligence	16
8	The five data moats (with replication math)	18
9	The integration plan — where every line of new code lives	20
10	Why no competitor can match — with stats	22
11	The 21-day sprint plan, day by day	23
12	Eval policy — the gate that ships or blocks every commit	25
13	Codebase organisation and folder layout	26
14	Commit and comment conventions	27
15	Day 1 — what we do tomorrow	28

0 · Executive verdict in one page

We have audited every layer of NyayaSathi (the citizen voice helpline) and Sanhita (the lawyer workbench). The decision is unambiguous and is the spine of this plan.

The decision

Integrate with Sanhita; do not rebuild. Sanhita's codebase has 7,839 lines of production-grade Python: the six-gate validator, the four-provider LLM router with circuit breakers, the BM25 retrieval index with atomic persistence, the Vault, the Workflows, the Realtime layer, and — crucially — a `nyaya_clients` table and `POST /api/nyaya/intake` endpoint that already anticipate exactly the handoff we need. We layer ~4,000 lines on top, not 12,000 lines from scratch.

Three things we got right and will defend

- The single source of truth for sessions on the NyayaSathi side.
- The smart-fallback that walks history when LLMs fail — keeps user-visible quality high under outage.
- The deliberate *refusal* posture in Sanhita's validator: refuses to write prose when it cannot ground.

Three things we will fix before launch

- PII at rest is plaintext today — must be AES-GCM encrypted before a real DV case lands.
- Distress detection is keyword-only and Hindi/English-only — paraphrase and 11-language coverage are blockers.
- Lawyer matching is mocked; real Sanhita-backed scoring with bar-council verification is the launch milestone.

Three sprints, each gated by 100+ question eval

Sprint 0 (week 1) — safety floor and bridge. Sprint 1 (week 2) — polish, female-voice auto-switch, real lawyer matching. Sprint 2 (week 3) — multi-modal vault and audit trail. Launch on Day 21.

The architectural promise (Section 4)

AI does **not** render judgment, does **not** interpret unmarked, does **not** take accountability. These three are the human contribution India lacks, and they are what we route through humans by design — enforced by validator gates G7 through G10 in code, not just policy.

1 · Sanhita codebase audit — integrate, don't rebuild

This section maps every component of the existing Sanhita codebase to a verdict: adopt as-is, extend, or replace. The total count of new code we have to write is the difference between the 'extend' rows and the 'replace' rows.

1.1 Inventory

Component	LOC	Quality	Verdict
validators/answer_gates.py (G1–G6)	209	Excellent	Adopt; port to NyayaSathi as JS
llm/router.py (4-provider chain)	477	Excellent	Adopt; replace homegrown JS cascade
retrieval_pkg/index.py (BM25)	294	Excellent	Shared retrieval backbone
validators/input_guards.py	226	Good	Adopt as shared module
agents/legal_agent.py	826	Solid	Reuse for Sanhita agent mode
brief_service.py (modes)	918	Solid	Reuse; add NyayaSathi voice mode
vault_service.py	289	Solid	Extend for audio/video/image
db_adapter.py (Postgres↔SQLite)	239	Solid	Adopt as shared persistence
realtime.py (socketio)	108	Good	Reuse; add nyaya:intake event
auth.py + nyaya_clients table	879	Already designed	Use exactly as is
/api/nyaya/intake (server.py L1857)	—	Wired	Just add HMAC verification
clients-pane.tsx (frontend)	—	Built	Reuse; add match button
eval/run.py + 60 prompts	—	Production	Extend to 250 prompts
BM25 corpus (15 MB pickled)	1,135 docs	Live	Grow to 110K via roadmap
Workflows: Draft, Review, Citator, Redline	442	Production	Extend with Compliance + Contract
Sarvam adapter (translate live)	180	Partial	Extend with stt() and tts()

1.2 What we delete from NyayaSathi

- lawyers.json (mock 10 lawyers) — Sanhita has the real table.
- lawyer-match.js (homegrown) — Sanhita's match engine takes over.
- Custom Promise.any LLM cascade — replaced by router pattern.
- File-backed case-store.js as primary store — Sanhita's nyaya_clients is canonical; NyayaSathi keeps a hot session cache only.

1.3 What we extend

- vault_service.py — adds audio (Sarvam STT), video (ffmpeg keyframes + STT), image (Tesseract + Gemini vision).
- validators/answer_gates.py — adds G7 no_judgment, G8 no_unmarked_interpretation, G9 no_accountability, G10 predictive_must_include_base_rate.
- workflows.py — adds Compliance Review and Contract Intelligence workflow types.
- llm/sarvam.py — adds stt() and tts() helpers used by NyayaSathi voice and Sanhita callbacks.

1.4 What we add fresh

- sanhita-client.js on NyayaSathi — HMAC-signed POSTer to /api/nyaya/intake.

- `lawyer_match.py` on Sanhita — scoring engine over a new `lawyers_profile` table.
- Multi-modal extractors module under `vault/extractors/`.
- Compliance edition primitives: `workspaces.py`, `policies.py`, `audit_chain.py`.
- Predictive surface (`predictive.py`) wired to outcome data once the flywheel turns.

The arithmetic that settles the debate

Building from scratch = ~12 weeks engineering + re-discovering 60 hand-tuned banned-phrase regexes + re-tuning circuit breaker thresholds + re-implementing dedup-by-case-id. Layering on top = 3 weeks engineering + tests catching what we change. There is no scenario in which from-scratch wins.

2 · The three-product reality

One platform, three editions. The same retrieval and validation infrastructure powers all three; each tier exposes additional surfaces and enforces additional contracts. The pricing reflects ascending value per seat — research alone is commoditised, real lead generation is rare, and accountable-by-design compliance is institutional.

Tier	Audience	Adds on top	Price / seat / month
NyayaSathi (B2C)	Citizens	Free voice helpline; routes 20% to Sanhita	— (free)
Sanhita Research	Solo advocate doing research	BM25 + memos + Court Search + Workflows	\$19
Sanhita Practice	Solo + small firm	Clients inbox (NyayaSathi leads), case mgmt, multi-tenant legal vault, paralegal automation	\$49
Sanhita Compliance	In-house counsel + compliance	Audit GRC, RBAC, policy engine, contract intelligence, spend risk	\$349

2.1 Why three editions, not one

A single SKU forces us to either over-charge solos or under-deliver to enterprise. A three-tier ladder with distinct value props per rung lets us address India's actual legal-services market structure — a long tail of solo practitioners and a short head of enterprise GRC teams who can pay 25× more for the right product.

2.2 The 18-month revenue ceiling

Tier	Target market in India	Realistic capture (18 months)	Monthly	ARR ceiling
Research \$19	200,000 paying advocates	2,000 seats	\$38,000	\$456,000
Practice \$49	50,000 solo + small firm	1,000 seats	\$49,000	\$588,000
Compliance \$349 (avg)	5,000 enterprise legal teams	200 seats	\$69,800	\$837,600
TOTAL		3,200 seats	\$156,800	\$1.88 M

These are realistic 18-month captures, not theoretical ceilings. The compliance tier alone roughly equals Practice + Research combined — and is where the moat sits because compliance buyers do not switch every quarter.

3 · The four customer journeys

Every product decision is justified by tracing it back to one of the four journeys. If a feature does not visibly help one of these flows, we do not build it now.

Journey A — Citizen, problem solved without lawyer (~70% of calls)

Dial 1800 → Hindi greet → DPDP consent → state problem → NyayaSathi RAG answers with statutory citation, actionable steps, helpline number → user hangs up satisfied.

Outcome: zero revenue, ₹6 cost per call. This is the moat-builder. We capture 1 M+ citizen-legal-event interactions per year that no competitor can buy.

Journey B — Citizen → matched to lawyer (~20% of calls)

5+ turn dialogue → AI offers lawyer → user accepts → budget question → POST /api/nyaya/intake (signed) → match engine ranks → user picks → SMS to lawyer → Exotel call-masking connects → recorded call → 7-day SMS outcome survey.

Outcome: lawyer pays Sanhita seat fee + commission. This funds NyayaSathi's runway.

Journey C — Solo lawyer using Sanhita Practice

Morning login → 3 new leads in inbox + 2 pending drafts + court calendar → open client → read intake summary + transcript + case-state JSON → 'Research' → 7-section memo with [n] cites → 'Draft Notice' → BNSS §482 anticipatory bail draft → upload client's voice note to Vault → ask Vault 'what date did the FIR say?' → answered with citation → send draft via WhatsApp → mark in_progress.

Paralegal work that used to take 4 hours, done in 30 minutes. \$49/month justified the moment a single matter is processed.

Journey D — Compliance team (e.g., bank's legal department)

GC creates 'Bank-X DPDP' workspace → configures policy: any clause referencing EU data must flag and route to Senior Counsel approval → junior associate uploads 50 vendor contracts → Sanhita extracts clauses, classifies risk, flags 12 → 12 flags routed to Senior Counsel queue with audit trail entry → Senior reviews, approves 8, escalates 4 → compliance report with hash-chain proof → quarterly DPDP audit: export full audit trail, prove every AI suggestion had a human checkpoint.

This is what makes the \$499 tier worth it. The product enforces what the user said India lacks — judgment, interpretation, accountability — by routing them to humans and proving cryptographically that it did.

5 · Multi-modal data: storage and upload

Real legal practice in India does not happen in clean PDFs. It happens in WhatsApp voice notes, photographed FIR copies, court hearing recordings, dashcam video, and screenshots of cyber-fraud chats. A product that only handles native PDFs is a toy.

5.1 What we receive

Modality	Source	Volume per case	Why it matters
Voice notes	WhatsApp from citizen	30s – 5 min	Tone matters, not just words
Hearing audio	Lawyer recording with consent	30 min – 6 hrs	What the judge said vs. what was minuted
Phone calls	Exotel call-masking	20 min – 1 hr	Both-side recorded
Video	Dashcam, CCTV, mobile	1 min – 1 hr	Accident reconstruction
Document photos	FIR, registry, deed (mobile)	1 – 20 pages	OCR'd citizen-side filing
Chat exports	WhatsApp, email backup	KB – MB	Cyber fraud evidence
Native PDFs	Bare acts, court orders, contracts	5 – 500 pages	Already handled

5.2 The pipeline

Upload → MIME router → modality-specific extractor → structured output → Sanhita's existing vault_chunks table (BM25 + Gemini embeddings).

```
audio/*    → Sarvam Saaras STT (22 langs) + diarization
video/*    → ffmpeg → keyframes (vision) + audio (STT)
image/*    → Tesseract + Gemini vision (layout-aware)
application/pdf → pdfplumber + layout extraction
text/*     → direct
```

ALL paths produce: {text_blocks, metadata, entities, pii_redacted, hash}
 Originals NEVER hit the LLM. Only redacted, structured extracts do.

5.3 Storage strategy

- **Tier-1 hot (Cloudflare R2 or AWS S3 IA, encrypted at rest):** originals; ~\$0.015/GB/month.
- **Tier-2 cold (Glacier Deep Archive after 90 days no access):** ~\$0.001/GB/month.
- **Hot extracts always live** in Postgres + pgvector. Originals retrieved by signed URL on demand.

5.4 The cost numbers

100 lawyers × 100 cases × 50 MB average = 500 GB hot = \$7.50/month. 1,000 lawyers × 100 cases = 5 TB hot = \$75/month. 5 TB cold = \$5/month. Storage is not the bottleneck; trust is.

5.5 Why this is hard to replicate

- Sarvam STT licensing — ~3× better than Whisper for Hindi/Tamil/Bengali. Indian advantage.
- Redaction-before-LLM is uncommon in legal-AI products; most ship raw transcripts to OpenAI/Anthropic. We stay DPDP-compliant by architecture.
- Tiered storage with structured-extract-as-source-of-truth keeps the long tail nearly free.

6 · The three stages of casework: Research → Analysis → Strategy

Mapping AI's role to each stage, with the AI-does-not-judge principle held strict.

Stage	AI role	Validation	Pricing tier
1 Research 'What does law say?'	Heavy. Hybrid retrieval + 7-section memo with [n] cited cases + 16-18 research points + multi-jurisdictional analysis.	31-66 hrs cash + 15-20 hrs review	\$150
2 Analysis 'How does law map to facts?'	Structured assistance. Fact extraction from vault, evidence correlation, legal reasoning (statutory, common law, equity).	61-99 hrs cash + 10-15 hrs review	\$190
3 Strategy 'What do we do?'	Minimal. Surface option set with base-rate, time, cost-benefit analysis.	71-99 hrs cash + 10-15 hrs review	\$190

This separation is also the billing structure. Research alone is commodity. Analysis needs case-state extraction and multi-modal vault. Strategy demands audit-trailed human checkpoints. Each tier earns its price.

7 · New surfaces: Predictive · Compliance · Contract Intelligence

7.1 Predictive analysis (case outcome probability)

Given case facts + jurisdiction + parties, returns probability of favourable outcome *with* the comparable past cases it is based on, median time-to-disposition, median cost, procedural roadmap. Output is never a single number alone — always paired with the comparables. Validator G10: predictive output must include base rate + sample size.

The data work is the moat: Indian outcome data is fragmented across HC websites, and aggregation + normalisation takes 6–12 months. Our outcome-SMS flywheel (7-day citizen survey post-handoff) is the only continuous source.

7.2 Compliance edition (the human-in-the-loop architecture)

Every AI suggestion in compliance mode passes through configurable human checkpoints with hash-chained audit trail.

```
Workspaces – tenant boundary (per-org corpus filter, policy set, members)
Roles      – Counsel · Senior Counsel · Compliance Officer · Auditor · Read-only
Policies   – declarative YAML, e.g.:
    policy: bank_x_dpdp
    triggers:
      - clause_type: data_processing
        jurisdiction: EU
        action: require_role: Senior Counsel
      - amount_inr: "> 1_00_00_000"
        action: require_role: GC
      - banned_clauses: [unlimited_indemnity, perpetual_license]
        action: hard_block
```

Checkpoints – every AI output flips `requires_human_signoff = true`; queued not delivered

Audit trail – append-only events with `sha256(prev_event)` hash chain

7.3 Contract intelligence (Indian-specific edge)

Most contract-AI (Harvey, Spellbook, Ironclad) is US/UK-trained. They get Indian Stamp Act, Companies Act, FEMA, RBI, GST implications wrong. We do not.

Layer	Output
Clause classification	Typed taxonomy: 'indemnity (limited)', 'IP assignment', 'termination for convenience'
IP conflict detection	Flags clauses contradicting prior agreements vault-wide
Indemnity analysis	Cap, scope, carve-outs vs. industry baseline
Limitation of liability	Enforceability across IN / SG / HK
Insurance fit	Required coverage given clause structure
Stamp duty	Jurisdiction-specific under Indian Stamp Act
Governing law / forum	Conflict-of-law issues, asymmetric forum risk

8 · The five data moats — with replication math

These are the assets that compound over time. A competitor with funding can copy any single one in months. Replicating all five together is an 18-month, \$700K+ exercise — and by then we are 18 months further ahead.

8.1 Moat-by-moat

Moat	Replication time	Replication cost	Compounds with...
Two-sided structure (NyayaSathi → Sanhita)	12–18 months	\$200K+	Citizen acquisition cost approaches zero
Outcome-data flywheel	Cannot shortcut	Must operate	Predictive surface accuracy
Indian NLP infrastructure	12 weeks	\$60K	Every quarter the gap widens
Multi-modal Indian legal vault	6 weeks	\$40K	Storage tail is nearly free
Hash-chained audit trail	4 weeks	\$25K	Compliance buyers do not switch
Eval-suite-as-asset	Cannot shortcut	Must operate	Every regression locks in quality
Sanhita's existing 7,839 LOC	4–6 months	\$250K	Already battle-tested

8.2 The Indian-NLP infrastructure budget breakdown

Asset	Build cost	Replication time
BNS↔IPC mapping table (600 sections)	1 week	Cannot shortcut — requires Indian legal context
Hindi legal tokenizer (compounds, sandhi)	2 weeks	Cannot shortcut
Citation parser (SCC, AIR, JT formats)	2 weeks	Cannot shortcut
OCR pipeline tuned for e-Courts PDFs	3 weeks	Cannot shortcut
Section renumbering migration (IPC → BNS, CrPc → BNSS)	1 week	Cannot shortcut
Helpline registry with quarterly verification	1 week + ongoing	Operational discipline
DLSA per-state referral table	2 weeks	Operational discipline

12 weeks of Indian-specific work that a foreign competitor cannot replicate without an India team.

9 · The integration plan — where every line of new code lives

9.1 On NyayaSathi

- `src/matching/sanhita-client.js` — HMAC-signed POST to Sanhita `/api/nyaya/intake` (~80 LOC)
- `src/validators/answer-gates.js` — port of Sanhita's G1–G6 (~250 LOC port)
- `src/llm/router.js` — port of Sanhita's 4-provider router (~400 LOC port)
- `src/voice/gender-detector.js` — F0 estimation; female-voice auto-switch (~150 LOC)
- `src/safety/distress-llm.js` — paraphrase classifier in parallel with keyword scan (~150 LOC)
- DPDP + recording disclosure first 10s of every call (~100 LOC)
- Distress copy translated to all 11 languages (data work)

9.2 On Sanhita

- HMAC verification middleware on `/api/nyaya/intake` (~30 LOC)
- `lawyer_match.py` — scoring engine over a new `lawyers_profile` table (~250 LOC + 1 migration)
- New endpoints: `/api/match/run` and `/api/match/accept` (~150 LOC)
- `vault/extractors/{audio,video,image}.py` — multi-modal pipeline (~500 LOC)
- `workspaces.py`, `policies.py`, `audit_chain.py` — compliance primitives (~800 LOC)
- `workflows.py` — Contract Intelligence workflow (~600 LOC)
- `predictive.py` — outcome probability surface (~400 LOC)
- `validators/answer_gates.py` — extend with G7 G8 G9 G10 (~150 LOC)

9.3 Shared infrastructure

- `llm/sarvam.py` — extend with `stt()` and `tts()` helpers
- Audit chain library reused by both products
- Eval harness — Sanhita's `eval/run.py` extended to cover NyayaSathi voice prompts

9.4 Total accounting

≈4,000 lines of new code on top of 7,839 lines of stable Python.

Three sprints. Three weeks. Eval-gated. Day-21 launch.

10 · Why no competitor can match — with stats

10.1 The aggregate replication math

Asset	Replication time	Replication cost
Two-sided structure	12–18 months	\$200K+
Three-edition tiering	6 months	\$150K
Outcome-data flywheel	Cannot shortcut	Must operate
Indian NLP infra (BNS map, citation parser, OCR, etc.)	12 weeks	\$60K
Multi-modal Indian legal vault	6 weeks	\$40K
Hash-chained audit trail (compliance)	4 weeks	\$25K
AI-does-NOT-judge architectural enforcement	4 weeks	Brand-defining
Eval-as-compounding-asset	Cannot shortcut	Must operate
Sanhita's existing 7,839 LOC	4–6 months	\$250K
TOTAL minimum to match	18+ months	\$700K+

10.2 The compounding argument

By the time a fast-follower has month-6 quality, we have month-24 quality, month-24 outcome data, month-24 eval suite, and month-24 Indian-corpus depth. The gap widens, it does not shrink, because every NyayaSathi call we take is a data point they cannot retroactively acquire.

10.3 What is uncopiable on principle

- The architectural commitment to refuse judgment: copying it requires giving up demos.
- The outcome flywheel: requires operating the helpline; cannot be bought.
- The eval suite as time-series: requires sustained operation; cannot be bought.

11 · The 21-day sprint plan, day by day

The shipping rhythm

Every change runs through the eval gate. Every fail becomes a permanent regression test. The eval suite never shrinks. Sprint targets are floors, not ceilings — we ship at $\geq$ target or we fix and re-run.

Sprint 0 — Bridge + safety floor (Days 1–7)

Day	What ships	Eval cut
1	Eval harness (CLI runner) + 30 baseline questions + first run against current NyayaSathi	baseline.json
2	HMAC bridge: NyayaSathi → Sanhita /api/nyaya/intake. sanhita-client.js. Add 20 questions	sprint0_d2.json
3	DPDP + recording + AI disclosure first 10s of every call. 11-language disclosure templates.	sprint0_d3.json
4	Distress copy translated to 11 languages + LLM tier-1.5 paraphrase classifier in parallel with	sprint0_d4.json
5	Sentry + safety-event file writer + Slack/email duty-officer alert pipeline.	sprint0_d5.json
6	Fix worst 5 fail patterns from D5 eval. Author 30 more questions covering safety paraphrases	sprint0_d6.json
7	Final Sprint 0 eval against 100 questions. Block if < 85%. Tag v0.5.0.	sprint0_final.json

Sprint 1 — Polish + real lawyer matching (Days 8–14)

Day	What ships	Eval cut
8	lawyers_profile schema + migration on Sanhita. Bar-council number field with verification step	sprint1_d8.json
9	lawyer_match.py with full scoring formula (specialty + lang + loc + budget + rating + outcome + gender-pref).	sprint1_d9.json
10	/api/match/run + /api/match/accept endpoints. Wire end-to-end into NyayaSathi.	sprint1_d10.json
11	Female-voice auto-switch (F0 + override + DV-force-female). Emotion-mapped TTS.	sprint1_d11.json
12	G7 G8 G9 banned-phrase extension. Cache-key fix (include case.type). PII at-rest encryption	sprint1_d12.json
13	Token-budget guard + extractor race-fix + buildSystemPrompt cleanup.	sprint1_d13.json
14	Final Sprint 1 eval against 150 questions. Block if < 90%. Tag v0.7.0.	sprint1_final.json

Sprint 2 — Multi-modal + audit + connection (Days 15–21)

Day	What ships	Eval cut
15	Multi-modal vault: audio (Sarvam STT), image (Tesseract + Gemini vision).	sprint2_d15.json
16	Video extractor (ffmpeg keyframes + audio STT). Storage tier-1/tier-2 wiring.	sprint2_d16.json
17	Hash-chained audit_events table + middleware. Workspaces + RBAC primitives.	sprint2_d17.json
18	Compliance policy YAML loader + checkpoint queue. G10 predictive-must-include-base-rates	sprint2_d18.json
19	Exotel call-masking. Lawyer SMS-to-accept (1=accept, 2=reschedule, 3=decline). 5-second	sprint2_d19.json
20	Per-state DLSA referral table. Right-to-erasure flow. End-to-end production smoke test.	sprint2_d20.json
21	Final eval against 250 questions. Block if < 92%. End-to-end smoke pass. Tag v1.0.0. LAUNCH	sprint2_final.json

12 · Eval policy — the gate that ships or blocks every commit

12.1 Question distribution (matches NALSA 2022 helpline call distribution)

Category	Count	Why this volume
FIR / police	25	28% of NALSA real-world calls
Family / DV / divorce / maintenance	15	16%; high distress-bypass rate
Property / land / rent / RERA	12	13%
Cheque bounce / fraud / cyber	10	11%; growing post-pandemic
Bail / arrest	8	9%; high-urgency
Salary / employment / labour	8	8%
Consumer / refund	7	7%
Motor accident	5	5%
Caste / SC-ST atrocity	5	Rural-rich, underserved
RTI / govt scheme	3	Niche but high-value
Distress / safety bypass	2	Tested separately — must pass at 100%

12.2 Block rules (non-negotiable)

Rule	Threshold	Action if violated
Distress questions pass rate	100%	Hard fail. No deploy.
Citation hallucination on any question	0	Hard fail.
Banned phrase ('as an AI', weasel) anywhere	0	Hard fail.
Safety-bypass time-to-first-byte	< 200 ms p95	Hard fail.
Overall pass rate	≥ 85% (S0), ≥ 90% (S1), ≥ 92% (S2)	Hard fail.

12.3 The regression rule

Every failure becomes a permanent regression test. The eval suite never shrinks; only grows. v2 = v1 + new regressions found in Sprint 0. By launch we have ~250 prompts in v4 and a 21-day time series of model behaviour saved as JSON.

13 · Codebase organisation and folder layout

As the codebase grows from ~3K to ~7K lines, structure prevents the kind of single-file monoliths (server.js at 2,400 lines today) that resist future maintenance. Discipline starts now.

13.1 Target structure

nyayasathi/	(was /Users/pranav/Desktop/n)
■■■■ README.md	
■■■■ docs/	
■ ■■■■ MASTER-PLAN.pdf	this document
■ ■■■■ ARCHITECTURE.md	one-page entry point
■ ■■■■ CONTRIBUTING.md	commit + comment conventions
■ ■■■■ CHANGELOG.md	per-sprint release notes
■■■■ src/	(Sprint 0 Day 5 cleanup)
■ ■■■■ server.js	route mounting + bootstrap only
■ ■■■■ routes/	per-endpoint files
■ ■■■■ case/	store · intent · slots · extractor
■ ■■■■ safety/	distress · helplines
■ ■■■■ matching/	sanhita-client
■ ■■■■ validators/	answer-gates · input-guards
■ ■■■■ llm/	router · gemini · sarvam · circuit-breaker
■ ■■■■ voice/	stt · tts · gender-detector
■ ■■■■ rag/	retrieval · citation-resolver
■ ■■■■ audit/	log · hash-chain
■ ■■■■ lib/	security · rules-engine
■■■■ public/	static assets
■■■■ data/	gitignored
■ ■■■■ cases/ sessions/ safety-events/	
■■■■ eval/	
■ ■■■■ v1/ questions.jsonl README.md	
■ ■■■■ harness.js	CLI runner
■ ■■■■ checks/	citations · banned · grounding · helplines · langua
ge	
■ ■■■■ reports/	versioned JSON, kept forever
■ ■■■■ fixtures/	mock audio · mock transcripts
■■■■ tests/	unit tests
■■■■ .gitignore package.json	

13.2 Migration sequence

- Day 1–4: leave src/ flat as-is; build new code into the target structure (eval/, src/safety/, src/matching/).
- Day 5 of Sprint 0: move existing files into target structure with git mv (preserves history). Update imports. Run eval to confirm no breakage.
- Sprint 1 onward: every new file lives in the target structure.

13.3 Why this layout

- Each folder maps to a section of this plan: a new contributor can find anything in <30 sec.
- Eval is a peer of src/, not buried — it is a first-class artefact, not an afterthought.
- Audit module is its own folder because compliance-edition demands cryptographic separation of concerns.

14 · Commit and comment conventions

14.1 Commit message format (Conventional Commits, lightly extended)

():

Refs: section 11 sprint 0 day 1
 Eval: 84% → 87% (sprint0_d2.json)
 Co-Authored-By:

14.2 Allowed types

Type	Use for
feat	User-visible new capability
fix	Bug fix
refactor	Internal restructuring without behaviour change
chore	Tooling, deps, config
docs	Documentation only
test	Eval question additions, unit tests
safety	Distress, helpline, DPDP, audit-trail changes (always reviewed)
data	Corpus or eval-data changes (additive only)

14.3 Allowed scopes

server · case · safety · matching · validators · llm · voice · rag · audit
 eval · docs · routes · vault · workflows · sanhita-bridge · contract-intel

14.4 Examples

safety(distress): translate critical helpline copy to 11 languages

Adds Bengali, Tamil, Telugu, Marathi, Gujarati, Kannada, Malayalam, Punjabi, Odia plus existing Hindi/English. Bengali DV victim was getting Hindi response before – empirically zero conversion to dialing 181.

Refs: section 5 (multi-modal), section 11 day 4
 Eval: 78% → 84% on distress subset (sprint0_d4.json)

feat(matching): replace lawyers.json with sanhita-client HMAC bridge

Deletes mock lawyers.json + lawyer-match.js. NyayaSathi now POSTs to Sanhita /api/nyaya/intake with HMAC-SHA256 signature. Sanhita's match engine returns ranked candidates from the lawyers_profile table.

Refs: section 9.1, section 11 day 9
 Eval: 87% → 89% (sprint1_d9.json)

14.5 Comment style for code

```
// Phase C.3: extracts entities asynchronously after the user-facing reply
// has shipped. Updates {entities, summary, urgency, needs_lawyer}. Skipped
// for ≤2-word messages (not worth the API call). 8s timeout – JSON-mode
// is sometimes slow on Flash; failures are caught up on the next turn.
```

- Block comments at the head of each module explain *why* the module exists, what it owns, and what it does NOT own.

- Inline comments mark non-obvious decisions and trade-offs (timeouts, retry policies, regex carve-outs).
- Never document *what* the code does at line level — the code says that already.

14.6 Pre-merge checklist

- Eval ran with pass rate $\geq$ sprint target.
- New failures added as regression tests in eval/v1/questions.jsonl.
- Conventional Commit message with Refs and Eval lines.
- CHANGELOG.md updated for the current sprint.

15 · Day 1 — what we do tomorrow

The first 24 hours

We do not write feature code. We build the gate that every future ship will pass through. By end of Day 1 we have an eval harness, 30 baseline questions, the first regression tests, and a baseline.json measuring exactly where we start.

15.1 Concrete actions

- Create eval/harness.js — CLI runner, ~150 LOC, POSTs to localhost:3000, captures, scores, saves report JSON.
- Create eval/v1/questions.jsonl with 30 hand-authored prompts (10 FIR, 8 family, 6 property, 4 cheque-bounce, 2 distress).
- Create eval/checks/ with 5 modules: citations.js, banned_phrases.js, helplines.js, grounding.js, language.js.
- Run baseline. Save eval/reports/baseline.json. Print summary.
- Pick the worst three fail patterns. File as Day 2 fix targets in CHANGELOG.
- Commit with message: test(eval): bootstrap harness + 30 baseline questions
- Tag commit: v0.4.0-baseline

15.2 Definition of done for Day 1

- `npm run eval` works from project root.
- Pass rate measured (we expect 50–65% baseline).
- Top three failure patterns named in writing.
- Day 2 plan reflects what the eval revealed, not what we assumed.

The promise we are making to ourselves

We do not lower the eval threshold to ship. We do not skip the regression test. We do not delete failing questions. The discipline of *not lowering thresholds* is the entire game — it is what separates a public-good service from yet another VC-funded chatbot.

End of plan. Re-generate after every sprint or whenever scope changes.